Hybrid GPU Cluster Design + Intelligent Job Router

Allan MacLean

1. Executive Summary

This design delivers a production-ready 512-GPU hybrid AI platform optimised for utilisation, latency, and fairness. It combines Slurm (for tightly-coupled distributed training) and Kubernetes (for inference and interactive workloads) on shared hardware using a deterministic coexistence strategy. The system sustains ≥500 GB/s aggregate storage throughput, supports 200+ concurrent interactive users, and elastically bursts to cloud when on-prem capacity is constrained. Had time permitted I would have spent more time investigating the Kubernetes Slinky operator to host Slurm as containers, which would allow one consistent platform to sweat the assets on.

Key differentiators

- Deterministic **scheduler coexistence** (no resource contention)
- **Dynamic node ownership** with fast drain/return (<60s target)
- **GPU fragmentation control** via MIG + bin-packing
- **Policy-driven routing** (latency, cost, queue depth)
- **Cloud bursting with guardrails** (cost caps + SLO triggers)

---

2. Requirements → Design Mapping

Requirement	Design Choice	Rationale
512 GPUs	64× nodes, 8× H100/A100 each	Standard SKU, failure domain isolation
200+ interactive users	K8s + Jupyter/VSCode	Elastic, multi-tenant
512-GPU training	Slurm partitions	Gang scheduling + MPI/NCCL
Low-latency inference	K8s Deployments + HPA/KEDA	Autoscale + rolling updates
≥500 GB/s storage	Lustre / GPFS over RDMA	Proven HPC throughput
Multi-tenancy	Slurm fair-share + K8s quotas	Hard + soft limits
AWS bursting	Slurm external partition	Native workflow continuity

3. High-Level Architecture

3.1 Compute

- 64 GPU nodes (8 GPUs/node), dual-socket CPUs, 1–2 TB RAM
- Node labels: `role=dual`, `fabric=ib,eth`, `gpu=mig-capable`

3.2 Control Planes

- **Kubernetes**: 3–5 control-plane nodes (HA etcd)
- **Slurm**: 1 controller + 1 backup + DB (MariaDB/Postgres)

3.3 Networking

- **InfiniBand (HDR/NDR, fat-tree)** for east-west training (MPI/NCCL)
- **Ethernet (100/200GbE, leaf-spine)** for control, storage, ingress
- RoCEv2 optional for storage acceleration

3.4 Storage

- **Lustre/GPFS** with 8–12 OSS/NSD nodes (NVMe)
- Target ≥500 GB/s aggregate read
- **Object tier** (S3-compatible) for model artefacts
- **Caching layer** on GPU nodes (NVMe local scratch)

4. Scheduler Coexistence (Critical Design)

Principle

At any moment, a node is owned by exactly one scheduler.

Mechanism (state machine)

- **K8s-owned (default)** → schedulable, device plugin active
- **Transition to Slurm**:
  1. Annotate node `owner=slurm`
  2. `kubectl cordon` + `drain --ignore-daemonsets --delete-emptydir-data`
  3. Disable GPU device plugin on node (label/taint)
  4. Slurm marks node `alloc` and runs job (exclusive GPUs)
- **Return to K8s**:
  1. Slurm job completes
  2. Re-enable device plugin
  3. `kubectl uncordon`
  4. Remove `owner=slurm`

SLOs

- Drain/return < 60s (no long-running pods on dual nodes)
- Zero double-scheduling (enforced by taints/labels)

GPU Sharing

- **Slurm**: full-GPU exclusive
- **K8s**: MIG profiles (e.g., 1g.10gb) + time-slicing for interactive

5. Workload Classes & Policies

Class	Platform	Policies
Training	Slurm	Min nodes, gang schedule, preemption tiers
Inference	K8s	HPA/KEDA, P95 latency SLO, canary deploys
Interactive	K8s	GPU time-slicing, idle cull, session caps

Preemption

- Interactive (low) < Inference (medium) < Training (high for reserved windows)

6. Multi-Tenancy & Fairness

Slurm

- Accounts per team
- Fair-share + QOS tiers (normal/high)
- GPU caps per account

Kubernetes

- Namespace per team
- `ResourceQuota` (GPU, CPU, memory)
- `LimitRange` defaults
- PriorityClasses for preemption

Example quotas

- vision-china: 128 GPUs
- nlp-us: 256 GPUs

7. AWS Bursting (Policy-Driven)

Trigger Conditions

- Queue wait time > X minutes (e.g., 30)
- Requested GPUs unavailable on-prem
- Cost budget remaining this period

Execution

- Slurm **external partition** mapped to cloud nodes
- AMI pre-baked with drivers + container runtime
- Data staged from object storage

Guardrails

- Max concurrent cloud GPUs per team
- Budget caps per month
- Automatic scale-down on idle

8. Observability & SLOs

- Metrics: GPU util, queue time, job success rate, pod latency
- Logs: centralised (Loki/ELK)
- Traces: inference paths (optional)

Alert (example)

```yaml
- alert: LowGPUUtilization
  expr: avg(gpu_utilization) < 0.35
  for: 30m
  labels:
    severity: warning
```

9. Security

- RBAC (least privilege)
- NetworkPolicies (namespace isolation)
- Image provenance (signed images)
- Secrets management (KMS/Vault)
- Slurm user isolation + cgroups

10. Cost Model (Ballpark)

Component	Estimate
GPU Nodes	$16M–$20M
Networking	$4M–$6M
Storage	$2M–$4M
Total	$22M–$30M

Intelligent Job Router (Production-Ready CLI)

Features

- YAML-driven submission
- Policy-based routing (type, queue depth, cost)
- Quota enforcement
- Slurm + Kubernetes backends
- Structured output + errors

---

submit_job.py

```
import argparse
import yaml
from router import route_job


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", required=True)
    args = parser.parse_args()

    with open(args.config) as f:
        config = yaml.safe_load(f)

    result = route_job(config)
    print(result)


if __name__ == "__main__":
    main()
```

---

router.py

```
from quotas import check_quota
from slurm import submit_slurm, slurm_queue_depth
from k8s import submit_k8s
```

```python
def route_job(config):
    check_quota(config)

    jt = config["job_type"]

    # Advanced policy: burst if queue too long
    if jt == "training" and slurm_queue_depth() > 50:
        return {"decision": "burst-to-cloud"}

    if jt == "training":
        return submit_slurm(config)
    elif jt in ["inference", "interactive"]:
        return submit_k8s(config)
    else:
        raise Exception("Invalid job type")
```

---

quotas.py

```python
TEAM_QUOTA = {
    "vision-china": 128,
    "nlp-us": 256
}


def check_quota(config):
    team = config["team"]
    gpus = config["gpus"]

    if gpus > TEAM_QUOTA.get(team, 0):
        raise Exception(f"Quota exceeded for {team}")
```

---

slurm.py

```python
import subprocess
import re
```

```
def slurm_queue_depth():
    try:
        out = subprocess.check_output(["squeue", "-h"]).decode()
        return len(out.strip().splitlines())
    except Exception:
        return 0


def submit_slurm(config):
    cmd = [
        "sbatch",
        f"--gres=gpu:{config['gpus']}",
        f"--time={config['duration_hours']}:00:00",
        "--wrap",
        config["command"],
    ]

    result = subprocess.run(cmd, capture_output=True, text=True)

    match = re.search(r"Submitted batch job (\d+)", result.stdout)
    job_id = match.group(1) if match else "unknown"

    return {
        "backend": "slurm",
        "job_id": job_id,
        "expected_start": "queue-based",
        "status": "submitted",
    }
```

---

k8s.py

```
import subprocess
import uuid
import json


def submit_k8s(config):
    job_name = f"job-{uuid.uuid4()}"
```

```
    job = {
        "apiVersion": "batch/v1",
        "kind": "Job",
        "metadata": {"name": job_name},
        "spec": {
            "template": {
                "spec": {
                    "containers": [
                        {
                            "name": "job",
                            "image": config["image"],
                            "command": ["/bin/sh", "-c",
config["command"]],
                            "resources": {"limits": {"nvidia.com/gpu":
config["gpus"]}},
                        }
                    ],
                    "restartPolicy": "Never",
                }
            }
        },
    }

    with open("job.json", "w") as f:
        json.dump(job, f)

    subprocess.run(["kubectl", "apply", "-f", "job.json"])

    return {
        "backend": "kubernetes",
        "job_id": job_name,
        "expected_start": "near-immediate",
        "status": "submitted",
    }
```

Example Manifest

```
team: vision-china
user: john.doe
```

```yaml
job_type: training
gpus: 32
duration_hours: 24
image: registry.lenovo.com/pytorch:2.4-cuda12.1
priority: normal
command: python train.py
```

README (Key Points)

- `python submit_job.py --config job.yaml`
- Requires: Python 3.9+, kubectl, Slurm CLI
- Outputs JSON with backend, job_id, expected start

Final Notes

This solution prioritises **utilisation, determinism, and operability**. The coexistence model avoids scheduler conflict, while policy-based routing ensures the right workload lands on the right platform at the right time.